

8.2 回文与后缀结构

O. 前置知识

以下我们认为 $\mathcal{T}\text{ime}$ 是一个连续统，记本份讲稿为 $\mathcal{S}$ ，其笔者为 $\mathcal{W}$ 。在此语境下我们不加证明地给出：

- **Lemma:** $\forall t \in \mathcal{T}\text{ime}$ ，若在 t 时刻 $\mathcal{W}$ 对 $\mathcal{S}$ 有非平凡作用，
那么 $\exists A, B \in \mathcal{E}\text{vent}$, $A \neq B$ ，在 t 时刻，所有 $\mathcal{W} + C$ 与 AB^{-1} 的 Kronecker 积为 **0**，其中 C 是任意**正常数**。[1]

上述结论也可以被感性而不严谨地理解为：（理解可能需要一定想象力）

- $\mathcal{W}$ 的所有**通货膨胀率**游离于二者之间，且其极大子理想的**正反应速率**反比于 $\mathcal{S}$ 的摆长。

此外，我们易见 $\mathcal{E}vent$ 构成一个 Bilen 域。由 GS-LC 引理 (GenshinLaunch Lemma)，在承认"中国窄鲸乘猜想"成立后，立刻导出：

- **Theorem:** 若正则且半稳定的 $\mathcal{W}$ 在中央 C 上不可约，那么 $\mathcal{W}$ 是唯心的。 [2]

(在 $\mathcal{W}^\flat$ 和 $\mathcal{W}^\sharp$ 的类似性质至今仍是 open problem。)

- **Corollary:** JurCai 是唯心的，此外 JurCai 也是玩原神玩的。

因此 $\exists k \in \mathbb{N}, \forall x \in \mathfrak{D}, \bigoplus_{x \mapsto x^{-1}} x^k \notin \text{JurCai}$ ，其中 $\mathfrak{D}$ 表示所有 丁-扩张 构成的集合。

若上述命题对于 k 成立，则称此性质为其 k - **没丁性**。易证任意原环有至多 $\clubsuit_k$ 个 k - 没丁的超越子环。[3]

证明留给读者作为习题。

[1]: Zhiyuan Hou, *et al.*, private communication (with one of my friends, Z. Xie) on bed.

[2]: Houhua Zhang, unpublished used toilet paper. A scanned copy is recorded in
C:/Users/Administrator/Desktop/StudyingResources .

[3]: Yeheng Xu, deleted blogs named "傻逼浙江你妈死了" and "我是萌萌的二次元捏"
(in Chinese).

下面的讲稿大体同构于我初三时写的学习笔记，因此语文〇分。

借助于上面的知识，我们将可以更从容地应对下面的各种 typo 和 nonsense。

A. 回文自动机

本名是回文树（EER Tree, Palindromic Tree），由于其具有自动机结构又称回文自动机（Palindromic Automaton），是一种可以存储一个串中所有回文子串的高效数据结构，最初由 Mikhail Rubinchik 和 Arseny M. Shur 在 2015 年发表。

树结构

字符串的回文结构有一些优美的性质，比如：

- **长度为 n 的字符串的本质不同非空回文子串数量不超过 n 。**

为什么呢？考虑到新加进一个字符后，至多会增加一个与之前本质不同的回文子串。

因为，若有两个新的子串，则较短的一个一定在之前出现过。

我们把每个本质不同的非空回文子串看作一个结点，其父亲定义为去掉开头、结尾的回文串。

容易发现这样长度为 $1/2$ 的串没有父亲了，我们定义两个虚根解决这个问题。

它们分别称为**偶根**和**奇根**，长度定义为 0 和 -1 ，这样的两棵树被我们称为**回文树**。

其实和 Manacher 一样加 `#` 大概也是可以的，但是较为繁琐。

为了简洁地表达一个回文串，我们可以只取其后半部分。也就是说，我们将一个回文串从回文中心开始的后半部分定义为其**半串**。

本质地，**回文树就是所有半串构成的 Trie**。

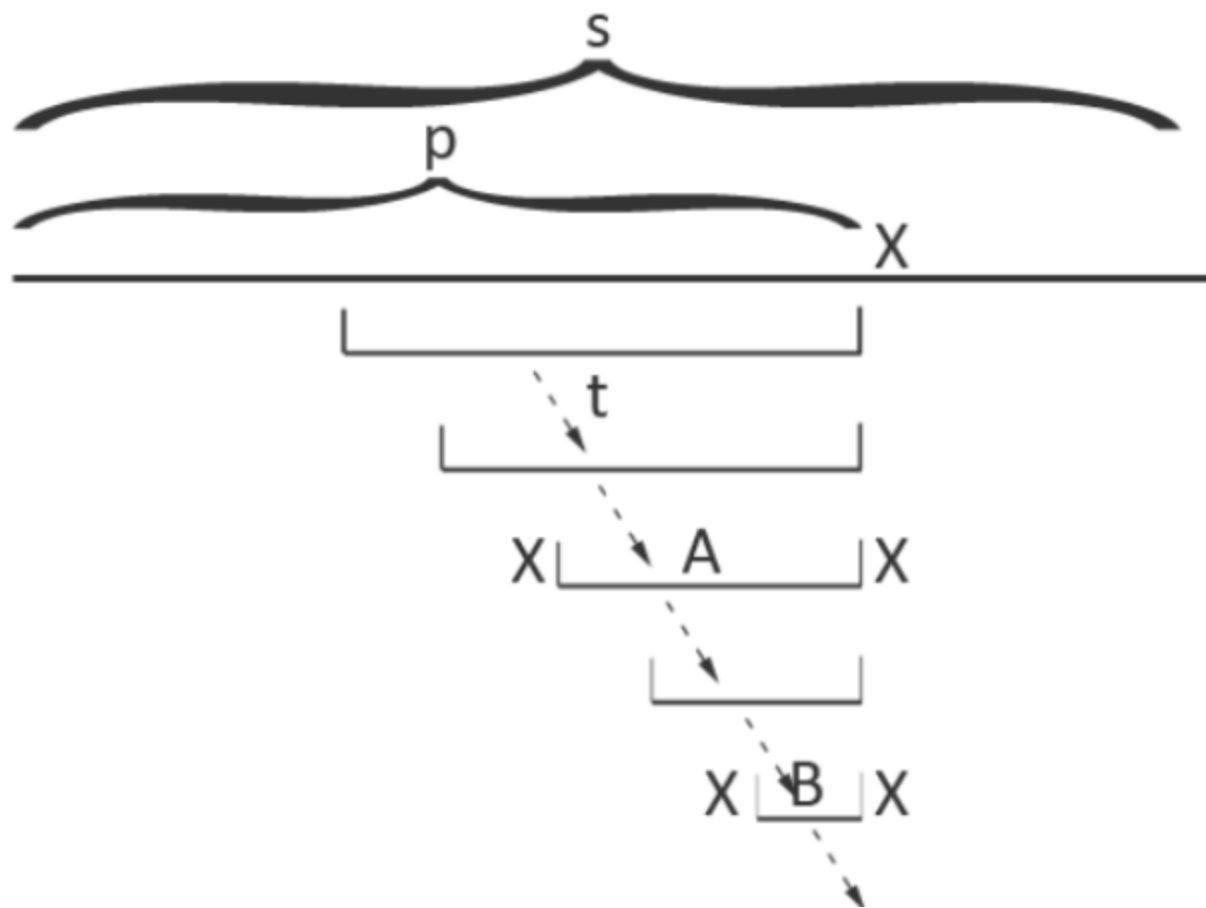
- 大多数情况下，即使题目中只使用到偶串信息，也不要尝试只建出偶串树，否则可能会出现一些奇怪的问题。

自动机结构

根据 ACAM 的经验，有了 Trie 自然要看看能不能搭建后缀结构。

我们同样采用**构造失配指针**的思路，但与 ACAM 不同的是，这个构造过程是在线的。

每次我们加入一个字符的时候，就和 KMP 类似地暴力往回跳，一直跳到匹配为止：（图中的 `XAX`）



（图源 OI - Wiki）

判断一下这个结点是不是新结点，如果是就新加一个结点，并寻找其失配指针。

具体地，我们从当前位置 **A** 继续往回跳，跳到下一个有匹配的位置 **XBX**，然后把失配指针连向它。

时间复杂度其实不难证是 $\mathcal{O}(n)$ 的，因为你跳过的每一条失配指针之后都不会跳到了，而失配指针的总数是 $\mathcal{O}(n)$ 的。

模板代码

可以参考代码看看具体实现细节。

```
struct PAM {
    int tr[N][26], len[N], dep[N], fail[N], tot;
    void init() { tot = 2, len[1] = -1, len[2] = 0, fail[2] = 1; }
    void insert(int &pos, int i) {
        while(s[i] != s[i - len[pos] - 1]) pos = fail[pos];
        if(tr[pos][s[i] - 'a']) return pos = tr[pos][s[i] - 'a'], void();
        int p = pos; pos = tr[pos][s[i] - 'a'] = ++tot, len[tot] = len[p] + 2;
        if(p == 1) return fail[pos] = 2, dep[pos] = 1, void();
        p = fail[p]; while(s[i] != s[i - len[p] - 1]) p = fail[p];
        fail[pos] = tr[p][s[i] - 'a'], dep[pos] = dep[fail[pos]] + 1; // 此时 p != 1
    }
} M;
```

具体实现时若 Trie 的空间代价无法接受，可以使用 `map` 或哈希表优化空间。

应用

PAM 有如下结构：

- 对于一个回文串，其对应结点在树上的祖先集合分别对应其所有回文后缀。

因此，PAM 的结构和 ACAM 是类似的，所以 ACAM 中能用的操作（如子树信息），对 PAM 依然适用。

回文子串匹配

给出两个字符串 S, T ，求最长公共回文子串的长度和数量。

我们建好 S 的 PAM，然后用 T 在上面跑回文子串匹配即可。

这里“匹配”的定义是对于每个前缀都求出它的最长回文后缀。

```
pos = 1; int ans = 0, cnt = 0;
fo(i, 1, m) {
    while(pos != 1 && (!tr[pos][t[i] - 'a'] || t[i] != t[i - len[pos] - 1]))
        pos = fail[pos];
    if(!tr[pos][t[i] - 'a'] || t[i] != t[i - len[pos] - 1]) pos = 2;
    else pos = tr[pos][t[i] - 'a'];
    // pos 就是 t[1...i] 的匹配
    if(len[pos] > ans) cnt = 1, ans = len[pos], mark[pos] = true;
    else if(ans == len[pos] && !mark[pos]) cnt++, mark[pos] = true;
}
```

另有一道经典题：

给定 n 个字符串 $S_1 \dots S_n$ ，求所有串的最长公共回文子串。

做法是找到长度最小的字符串 S_1 ，把他和后面每个字符串回文子串匹配。

这样可以找到 S_1 每个前缀对所有其他串匹配长度的最小值的最大值。

单次匹配复杂度是 $\mathcal{O}(|S| + |T|)$ 的，总复杂度依然是

$$\mathcal{O}(m|S_1| + \sum_{i=1}^m |S_i|) = \mathcal{O}(\sum |S_i|), \text{ 因为 } |S_1| \leq \frac{\sum |S_i|}{m}.$$

PAM 配合 DP

P4762 [CERC2014]Virus synthesis

初始有一个空串，你需要利用下面的操作构造给定串 S 。

1. 在串开头或末尾加一个字符；
2. 在串开头或末尾加一个当前串的反串；

求最小化操作数， $|S| \leq 10^5$ ，字符集为 $\{A, T, C, G\}$ 。

考虑我们最后一次进行第二种操作后的串 T ，显然它是 S 的子串，而且之后还需要进行 $|S| - |T|$ 次操作。

这样我们只需要对每个偶回文串，计算最少几次操作能够得到它即可。

不难想到可以 DP，显然转移顺序是从短到长的，所以我们需要 BFS。

更具体地，我们 BFS 回文树，在 BFS 的过程中在 PAM 上转移，形式是：

$$f_x = \min_{y \subseteq \text{semi}(x)} f_y + \frac{|x|}{2} - |y| + 1$$

即：

$$f_x - |x| = \min_{y \subseteq semi(x)} \{f_y - |y|\} - \frac{|x|}{2} + 1$$

其中 $semi(x)$ 是半串，这可以在 fail 树上维护：只需在每个结点记录一个指针，指向最近的一个 $len_f \leq \frac{len_x}{2}$ 的偶回文点。

PAM 的双侧插入

有个小 trick 就是 PAM 可以支持双侧插入，维护两个指针即可，复杂度易证也是线性。
只需注意通到另一端了把另一端也更新一下。

回文串划分问题

给定一个字符串 S ，问有几种 $s_1 \dots s_k$ 使得 $s_1 + s_2 + \dots + s_k = S$ 且所有 s_i 非空且都为回文串。

(求 k 的最小值或者要求 s_i 均为偶回文串在这里是等价的。)

我们考虑 DP，设 dp_i 是 $S[1 \cdots i]$ 的答案数，转移可以直接枚举所有当前的回文后缀转移过来，这是 $\mathcal{O}(n^2)$ 。

上节课讲过，这些回文后缀对应了回文 Border，自然也有 $\mathcal{O}(\log n)$ 个等差数列的性质。

我们考虑找到一种复杂度依赖于等差数列个数的做法。

对于每个等差数列，设其公差为 d 。考虑其在 $i - d$ 处的贡献，可以发现二者之差至多包含一个位置，直接转移即可。

具体细节可见 [OI-Wiki](#)。

```
int pos = 1; dp[0] = 1;
fo(i, 1, n) {
    insert(pos, i); if(pos == 1) continue;
    for(int j = pos; j > 2; j = jump[j]) {
        g[j] = dp[i - len[jump[j]] - diff[j]];
        if(fail[j] != jump[j]) g[j] = (g[j] + g[fail[j]]) % P;
        if(i % 2 == 0) dp[i] = (dp[i] + g[j]);
    }
} printf("%lld\n", dp[n]);
```

B. 后缀数组

算法过程

后缀数组 (Suffix Array) : 把所有的 n 个后缀按照字典序排序后的序列。

考虑采用一个倍增的思路, 每次对所有的 $S[i \cdots i + 2^w - 1]$ 排序。

怎么从 w 转移到 $w + 1$ 呢? 其实不难, 以 (rk_i, rk_{i+2^w}) 为二元组进行排序即可。

暴力 `sort` 是 $\mathcal{O}(n \log^2 n)$ 的, 可以基数排序优化成 $\mathcal{O}(n \log n)$ 。

实现

可以对着代码看一下细节：

```
struct Suffix_Array {
    int sa[N], rk[N], cnt[N], id[N], ork[N << 1];
    void build() {
        int m = (1 << 7), p = 0; // m = |Sigma|
        fo(i, 1, n) cnt[rk[i] = s[i]]++;
        fo(i, 1, m) cnt[i] += cnt[i - 1];
        fr(i, n, 1) sa[cnt[rk[i]]--] = i;
        for(int w = 1; w <= n; w <= 1, m = p, p = 0) {
            fo(i, n - w + 1, n) id[++p] = i;
            fo(i, 1, n) if(sa[i] > w) id[++p] = sa[i] - w;
            p = 0, cl(cnt, 0), memcpy(ork, rk, sizeof(rk));
            fo(i, 1, n) cnt[rk[id[i]]]++;
            fo(i, 1, m) cnt[i] += cnt[i - 1];
            fr(i, n, 1) sa[cnt[rk[id[i]]]--] = id[i];
            fo(i, 1, n) rk[sa[i]] = (ork[sa[i]] == ork[sa[i - 1]] &&
                ork[sa[i] + w] == ork[sa[i - 1] + w] ? p : ++p);
            if(p == n) break;
        }
    }
} SA;
```

解释一下里面的数组：

- `sa[i]` 是前一次的后缀数组；
- `rk[i]` 表示 i 后缀的排名（即最后有 `sa[rk[i]]==i` 成立）；
- `cnt[i]` 是排序用的桶；
- `ork[i]` 是前一次的排名，**为避免越界要开到** `ork[N << 1]`；
- `id[i]` 是第二关键字排序的数组。

然后是第二关键字无需基数排序，可以先把空串塞进数组，再按序塞进去。

height 数组

下面是一个 SA 里面非常重要的概念，称为 height 数组（下写为 $ht(i)$ 或 ht_i ）。它定义为：

$$ht(i) = \text{LCP}(sa_i, sa_{i-1})$$

这个东西有一个简单的 $\mathcal{O}(n)$ 做法，利用一个性质：

$$ht(rk_i) \geq ht(rk_{i-1}) - 1$$

感性理解一下，就是 $i - 1$ 和 $sa_{rk_{i-1}-1}$ 匹配到的位置，必然 i 也能和 $sa_{rk_{i-1}-1} + 1$ 能匹配到。

这样就至少是 $ht(rk_{i-1}) - 1$ 了。势能分析一下复杂度显然是 $\mathcal{O}(n)$ 。

```
fo(i,1,n) {  
    ht[rk[i]] = max(0, ht[rk[i - 1]] - 1);  
    while(s[i + ht[rk[i]]] == s[sa[rk[i] - 1] + ht[rk[i]]])  
        ht[rk[i]]++;  
}
```

应用

利用 ht 数组可以做很多事，比如：

- 求 $\text{LCP}(S[x \cdots n], S[y \cdots n])$ 。

答案是 $\min_{rk_x+1 \leq i \leq rk_y} ht_i$ ，可以用 ST 表维护，复杂度 $\mathcal{O}(n \log n) - \mathcal{O}(1)$ 。

Hint：利用这个性质可以用单调栈 / 笛卡尔树结构求一些东西，如求

$$\sum_{x=1}^n \sum_{y=1}^n \text{LCP}(S[x \cdots n], S[y \cdots n])。$$

- 求本质不同子串个数

答案是 $\binom{n}{2} - \sum_{i=2}^n ht_i$ 。

- 求多个字符串的最长公共子串。

可以和求回文的做法一样，用一个最短的字符串去做。

也可以把所有字符串放到一起建 SA，然后双指针。

例题

- 经典问题

给定一个字符串 S 和一个初始为空的串 T ，每次可以选择 S 的开头或末尾，将其删除并加入 T 的末尾。

要求 T 字典序最小， $|S| \leq 5 \times 10^5$ 。

容易想到一个贪心：每次选首 / 尾中较小的一个。

如果一样怎么办呢？其实如果只能选一个（即选另外一个一定会导致答案更劣），肯定是连着选一段，这可以后缀排序实现。

- P1117 [NOI2016] 优秀的拆分

给定一个长度为 n 的字符串 S ，取出一个连续子串，如果这个子串能被写成 AABB 的形式则称为一个优秀的拆分。不同位置或拆分方式不同算作不同的优秀的拆分。求整个字符串中连续子串不同的优秀的拆分的个数。

多测， $T \leq 10, n \leq 3 \times 10^4$ 。

有一个 $\mathcal{O}(n^2)$ 的无脑 hash 暴力能拿 95pts，我觉得出题人准有点大病。

这个题主要是介绍一种常见套路：**设置观察点**。

对于长度为 x 的 AA 型的字符串，我们每隔 x 的长度设立一个观察点。

不难发现每个 AA 型的会经过两个观察点，而且被分成观察点前的一段前缀和一段后缀。

这样我们可以用 SA 求 LCP / LCS，再差分去区间加。

复杂度是调和级数，就是 $\mathcal{O}(n \log n)$ 。

- P7361 「JZOI-1」拜神

给一个串， Q 次询问区间 $[l, r]$ 中至少出现两次的子串的最大长度。

$1 \leq n, q \leq 10^5$ 。

这里有第二个重要技巧：**将 LCP 关系转化成连通块。**

我们把每个 ht_i 看作 (sa_i, sa_{i-1}) 的一条边权为 ht_i 的边，然后和 Kruskal 一样降序排序，用并查集维护连通块。

这样同一个连通块里的就是 $LCP \geq x$ （当前边权）的。还可以配合启发式合并实现别的功能。

这个题就可以对每个 r 维护主席树，记录最大的 l 使得 $LCP(S[l \cdots n], S[r \cdots n]) \geq x$ ，这样每次询问就可以二分然后查询区间 max。

C. 后缀树

后缀树子串结构

后缀树 (Suffix Tree)：考虑对一个字符串 S ，把所有后缀 $S[1 \cdots n]$ 插入到一棵 Trie 中去。

直接插进去显然结点数可以达到 $\mathcal{O}(n^2)$ 级别，考虑优化：只需把树上的所有**二度点**删去。换言之，就是把所有只有一个儿子的点删去，查询的时候按着原先的路走，但是可能会走到路中间。

这棵树相当于所有叶子结点的虚树，这样后缀树的结点数可以证明至多有 $2n + \mathcal{O}(1)$ 个。

但是这样会带来一个问题：后缀点不一定在后缀树上标出来！

其实这是容易解决的：最后再插入一个特殊符号，比如 `#`。

发掘一下这棵树的一些性质：

- 一个字符串能被 S 的后缀树接受，当且仅当它是 S 的一个子串；
- 如果当一个字符串走到路中央时我们钦定它把路走完了，那么对于每个结点，能走到它的所有字符串构成一个等价类。

我们称它为 **Startpos 等价类**，因为同一个等价类在 S 中的所有出现的**起始位置集合**都相同。

事实上，**Startpos 集合**就是子树中后缀结点对应的后缀组成的集合。

另外，等价类中最长的那个字符串被称为这个等价类的**代表元**。

- 进一步地我们考虑不同等价类间 Startpos 集合的关系。我们发现：

在后缀树上呈祖先关系的两个点的 Startpos 集合呈包含关系，不呈祖先关系的 Startpos 集合一定不交。

这样我们找到了**利用后缀树的字符串子串结构**：

- 字符串的所有子串按照 Startpos 集合被分成 $\mathcal{O}(n)$ 个等价类。

任意两个等价类的 Startpos 集合呈包含或者不交的关系，构成一个树形结构。

同时还可以定义**广义后缀树**（Generalized Suffix Tree）为把一组字符串中的所有后缀插入的结构，实现方式是把它们通过若干个不同的分割符连起来然后再建树。

Ukkonen 算法

问题来了，怎么快速构建一棵后缀树呢？

我们可以使用 Ukkonen 算法在**线性时间内在线**地构建出后缀树。~~不过可能全中国会 Ukkonen 算法的人都非常少。~~

具体描述可以看[这里](#)或[这里](#)，以下有一些简要的描述。

先给代码。

```

int len[N], st[N], fail[N], tr[N][30], tot;
int cur, pos;

inline void init() { tot = pos = 1, len[0] = inf; }
int New(int S, int L) {
    tot++, st[tot] = S, len[tot] = L; return tot;
}
void insert(int x) {
    cur++; // 1
    for(int last = 1; cur; ) { // 5
        while(cur > len[tr[pos][((int)s[x - cur + 1])]]) // 2
            cur -= len[pos = tr[pos][((int)s[x - cur + 1])]];
        int &v = tr[pos][((int)s[x - cur + 1])];
        char nxt = s[st[v] + cur - 1];
        if(!v || s[x] == nxt) {
            fail[last] = pos, last = pos;
            if(!v) v = New(x, inf); // 4.a
            else break; // 3
        } else { // 4.b
            int u = New(st[v], cur - 1);
            tr[u][((int)nxt)] = v, tr[u][((int)s[x])] = New(x, inf);
            st[v] += cur - 1, len[v] -= cur - 1;
            fail[last] = v = u, last = u;
        }
        if(pos > 1) pos = fail[pos]; else cur--;
    }
}

```

我们把每条边用一个二元组 (st, len) 记录在儿子上，表示这条边上的内容是 $S[st \cdots st + len - 1]$ 。

边上的字符串在插入一个 x 时可能可以**自然延伸**，所以有时 $len = \infty$ 。

我们维护一个二元组 (pos, cur) 表示当前的**最长隐后缀**（最长的出现了至少两次的后缀，对应到后缀树上就是"在某条边的中间"或者"在当前某个非叶子结点"），这个二元组的意思是**最长隐后缀**从 pos 走出了 cur 的长度。

1. 每次插入时我们让 *cur* 自增 1，表示长了一位。
2. 如果超过了这条路的长度就往后跳。
3. 如果当前插入的这位和这条路上的下一位 *next* 相等，那么表示最长后缀依然是最长的，直接 break；
4. 否则有两种情况：
 - a. $v = 0$ ，路没了。这时我们新建一个叶子挂在后面。
 - b. 否则就是要从路的中间分岔了。先新建一个分岔点，再挂一个新叶子。

5. 但是还没完！要是我们成功插入了长为 L 的最长隐后缀，那么 $L - 1$ 同样也是隐后缀！

为什么呢？注意到隐后缀本质上就是出现了至少两次的后缀，所以所有 $x \leq L$ 都是隐后缀。

这样如果插入成功， L 不再是最长隐后缀，我们需要检查 $L - 1$ 能否被插入。

具体地，我们在每个非叶子结点上维护一个 *fail* 指针指向其最长真后缀。

实现时，我们记录一个 *last* 表示上一次访问到（新建）的结点，然后把它的 *fail* 指针挂到当前点上。

特别地，若 $pos = 1$ ，那么只能通过 *cur* 自减来完成。

细节看代码。

与 SA 的关系

考虑一下后缀树和后缀数组的关系：

- sa 数组就是后缀树上叶子结点的 DFS 序。
- ht 数组就是后缀树上相邻两个叶子结点 LCA 的高度。

可见，后缀数组只保留了后缀树的一些信息，是后缀树的真子集。

例题

核心就是一句话：**Startpos 集合就是子树中叶子结点所对应的后缀点组成的集合。**

- P3804 【模板】后缀自动机 (SAM)

给定一个只包含小写字母的字符串 S 。

请你求出 S 的所有出现次数不为 1 的子串的出现次数乘上该子串长度的最大值。

$$1 \leq |S| \leq 10^6。$$

建出后缀树，然后求每个子树中的叶子结点个数。

- P1117 [NOI2016] 优秀的拆分

给定一个长度为 n 的字符串 S ，取出一个连续子串，如果这个子串能被写成 AABB 的形式则称为一个优秀的拆分。不同位置或拆分方式不同算作不同的优秀的拆分。
求整个字符串中连续子串不同的优秀的拆分的个数。

多测， $T \leq 10, n \leq 3 \times 10^4$ 。

我们考虑不设关键点怎么做这个题。

先把问题转化成数有几个 i 作为 AA 串的 Startpos, 即:

$$\sum_{j=i+1}^n [\text{LCP}(i, j) \geq j - i]$$

我们把 $\text{LCP}(i, j)$ 看作 $\text{dep}(\text{LCA}(i, j))$, i, j 也相应化成 dep , 得到:

$$\sum_{j=i+1}^n [\text{dep}(j) - \text{dep}(\text{LCA}(i, j)) \leq \text{dep}(i)]$$

可以使用 DSU on tree 维护动态开点线段树, 算双向的贡献即可。

- P4094 [HEOI2016/TJOI2016]字符串

给出串 S ， Q 次询问，每次询问给出 a, b, c, d 四个参数，问子串 $s[a..b]$ 的所有子串和 $s[c..d]$ 的最长公共前缀的长度的最大值。

$1 \leq |S|, Q \leq 10^5$ 。

做法是二分一个长度 L ，然后判断是否有 $\max_{a \leq i \leq b-L+1} \text{LCP}(c, i) \geq L$ 。

这样我们把问题转化为：

每次给定一个字符串 S 为 c 的长为 L 的前缀，然后给定 $[l, r]$ ，询问 S 是否至少是 $Suf_{l \dots r}$ 中一个字符串的前缀。

考虑建出后缀树，问题转化为子树内是否有 $[l, r]$ 区间内的点。这可以主席树维护。

D. 后缀自动机

fun fact: 我们的 cjn 先生发明了广义前缀自动机!!!

SAM 的概念

后缀自动机 (Suffix AutoMaton) 定义为能接受一个串 S 的所有子串的状态数最小的自动机。

我们希望它应当满足如下性质：

- 状态数和转移数都是线性的，便于处理；
- 其接受且仅接受 S 的所有子串，因此它应当是一张 DAG，且任意子串唯一对应一条路径。

怎么去构建它呢？由 Nerode 定理，一个自动机最小当且仅当**不可区分等价类的数量**恰与 **DFA 状态数**相等。

换言之，我们只需要保证 SAM 中每个状态代表了一个等价类。但是这是关于什么的等价类？

注意到这个不可区分等价类的定义是：

- 对任意 $x, y, z \in \Sigma^*$ ，若 xz, yz 同时属于或不属于 Σ^* ，则称 x, y 属于同一不可区分等价类。

类似后缀树地，我们把 $\text{endpos}(T)$ 定义为 T 在 S 中的所有出现的末尾位置。

我们证明这个不可区分等价类在 $\Sigma^* = \text{substr}(S)$ 时就是 endpos 等价类！

首先，同一个 endpos 等价类肯定在一个不可区分等价类中。

反过来，如果两个串在不同的 endpos 等价类中，那么随机选一个 endpos 出来取下一个字母 z 。

如果 xz, yz 当前的 endpos 相同（显然不为空），则说明 x, y 的 endpos 集合有交。

由后缀树里推出的东西，二者呈包含关系，取补集中的一个字母即可。

这样 xz, yz 的 endpos 不同，不断加下去一定能搞出矛盾。

这样我们已经刻画了 SAM 每个状态的形态：一个 endpos 等价类。

事实上这也说明了：从初始状态出发的每一条路径**唯一对应**一个子串。

类比后缀树的结构，我们可以把所有状态写成一棵树的形式。祖孙包含，其余不交。

此外，每个等价类中长度不重且连续、彼此呈后缀等性质也可以直接应用。

这棵树被我们称为 link 树，父亲结点被称为 $\text{link}(x)$ 。它与反串的后缀树等价。

这说明：**SAM 中状态数是 $2n + \mathcal{O}(1)$ 的。**

但是我们要的不只是这个 link 树，还要自动机结构。

Blumer 算法

建出 SAM 需要 **Blumer 算法**。

先定义一个辅助概念 $\text{len}(x)$ 表示该等价类中最长字符串的长度，同理定义 $\text{minlen}(x)$ 。

推几个关于 len 的结论。

- 对所有的转移 $p \rightarrow q$, 有 $\text{len}(p) + 1 \leq \text{len}(q)$ 。

找一个最长的即可。

- 对非初始状态 q , 恰好存在一个转移 $p \rightarrow q$ 使 $\text{len}(p) + 1 = \text{len}(q)$ 。

注意, 反过来对 p 是不对的。

显然。类似地可以把 len 换成 minlen 。

另外, 这个唯一的 p 是所有别的转移 $p' \rightarrow q$ 中 p' 在 link 树的后代。

因此所有转移构成 link 树上 p_{minlen} 到 p_{len} 的一条祖先链。

与后缀树类似，我们采用在线的增量法构造 SAM。

插入一个 s_i 时，我们执行如下算法：

- 由于 $s_{1\dots i}$ 必然新构成一个等价类，那么我们新建一个结点 cur ， $\text{len}(cur) = i$ ；
- 记 $s_{1\dots i-1}$ 位于的等价类为 las ，那么新建转移 $\delta(las, s_i) \leftarrow cur$ 。
- 从 las 不断沿着 link 树往回跳，设跳到 p 。如果 $\delta(p, s_i)$ 不存在，那么说明之前没有出现过 $p + s_i$ 的形式。

那么我们直接 $\delta(p, s_i) \leftarrow cur$ 。

- 如果跳到根了还没有跳到存在 $\delta(p, s_i)$ 的位置，那么说明 s_i 是第一次出现，直接 $\text{link}(cur) \leftarrow rt$ ，退出算法。

- 否则设最近的那个是 $\delta(p, s_i) = q$ ，分类讨论：
 - 若 $\text{len}(q) = \text{len}(p) + 1$ ，则说明在 p 的代表元后面加一个 s_i 就是 q 的代表元，我们直接让 $\text{link}(cur) \leftarrow q$ 。
 - 否则，举一个例子 **abcb** 插入 s_4 ，此时 $p = \epsilon, q = \mathbf{ab}$ ，看起来是要分裂状态了。

考虑新建一个结点 c 为 $s_1 \dots_i$ 和 q 的最长公共后缀，**将其转移指针从 q 处复制过来**，并且 $\text{link}(q) \leftarrow c$ 。

这样我们就可以找到 $\text{link}(cur) \leftarrow c$ 了，而且 $\text{link}(c) = \text{link}'(q)$ ，即原来的最长后缀，看起来非常漂亮。

但是还有转移指针需要更新。

我们从 p 开始往上跳，若 $\delta(p, s_i) = q$ 那么我们重新指 $\delta(p, s_i) \leftarrow c$ 。

这样插入就完成了。


```

int tr[N][S], lnk[N], len[N], tot;
void insert(int &las, int i) {
    int p = las, cur = ++tot; len[cur] = i, las = cur;
    while(p && tr[p][(int)s[i]] == 0) tr[p][(int)s[i]] = cur, p = lnk[p];
    if(p == 0) return lnk[cur] = 1, void();
    int q = tr[p][(int)s[i]];
    if(len[q] == len[p] + 1) return lnk[cur] = q, void();
    int c = ++tot; memcpy(tr[c], tr[q], sizeof(tr[c])); // !
    len[c] = len[p] + 1, lnk[c] = lnk[q], lnk[cur] = lnk[q] = c;
    while(p && tr[p][(int)s[i]] == q) tr[p][(int)s[i]] = c, p = lnk[p];
}

```

广义 SAM 的实现是对所有串建一棵 Trie，每次将 *las* 标为父结点的 *cur*，不再展开。

接下来我们证一下几个复杂度，不管后面的常数了：

- 状态数不超过 $2n + \mathcal{O}(1)$ 。

上面证过了。

- 转移数不超过 $3n + \mathcal{O}(1)$ 。

首先 $\text{len}(p) + 1 = \text{len}(q)$ 的转移只有 $2n + \mathcal{O}(1)$ 条。

对于其余的转移 (p, q) ，我们找出 p, q 往前、往后的最长路径，显然这代表一个后缀。

（注意，不是所有后缀都是以这个形式出现，但是以这个形式出现的一定是后缀。这样构造只是为了便于利用唯一性说明。）

由于后缀只有 $n - 1$ 个，且**不同路径得到的字符串不同**，所以其余的转移至多 $n - 1$ 条，加起来是 $3n + \mathcal{O}(1)$ 。

- 算法的时间复杂度是 $\mathcal{O}(n|\Sigma|)$ 的，其中除代码中 `!` 那一步外都是线性。

分析代码，不新建转移或结点的过程只有最后一行的重指指针为 c 的过程，可知其余部分都是 $\mathcal{O}(n)$ 的。

对这部分复杂度的一个简单证明是：每次重新赋值的一段都是从 c 到 $\text{link}(c)$ 对应到 p 祖先链的一段。

由于这个 $\text{link}(c)$ 其实就是现在的 $\text{link}(\text{link}(las))$ ，因此每次的 $\text{link}(\text{link}(las))$ 深度都会变小。

更重要地，画个图可以知道，每次幅度和上面那个过程只有一个常数的差距。

因此势能分析一下就是 $\mathcal{O}(n)$ 的了，详细过程不再给出。

分析

三种后缀结构的关系：

后缀数组的信息包含于后缀树，而后缀树又隐含在后缀自动机里。

所以后缀自动机是三种后缀结构中最强大的，其实代码也挺好写。

这三者好似一个反比例函数，功能依次递增，直观性、便捷性依次递减。

SAM 和 ACAM 的关系：

SAM 其实就是所有后缀的 ACAM 的**压缩形式**。

怎么压缩的呢？就是把 fail 树中的二度点全部压缩成一个点。

这样 Trie 也自然变成了 DAG。

简单结论

- 本质不同子串数： $\sum \text{len}(i) - \text{len}(\text{link}(i))$ 。
- 最长后缀匹配：给定 S, T ，可以求出 T 的每个前缀作为 S 子串出现的最长后缀。
只需在 SAM 上跑的时候维护当前串的长度即可。
- 广义 SAM (Generalized SAM)：参考[这篇讨论](#)，一般情况下**最好 BFS 建**，否则容易寄。

- endpos 集合：后缀树里的 startpos 集合就是子树内所有后缀点，后缀自动机里的 endpos 呢？

其实 endpos 集合就是 link 树的子树内所有代表前缀的点，插入时顺便 mark 一下就好了。

有些题同样要与线段树合并配合，可以采用可持久化线段树合并的方式维护。

- 定位子串（给定 l, r ，求 $s_{l \dots r}$ 的状态）：从 pos_r 往上倍增找到 $\text{len} = r - l + 1$ 。
- $|\Sigma|$ 很大的时候：用 `map` 维护转移，由转移的线性性，复杂度 $\mathcal{O}(n \log |\Sigma|)$ 。
（使用哈希表更优）

- 区间子 SAM：就是让我们刻画 $S[l \cdots r]$ 的一个 SAM 或类似结构。

实际上可以直接用整串的 SAM，但是有两个问题需要解决：

- 自动机结构上，我们需要判断一个等价类 p 是否依然存在。

考虑线段树二分找到 $\text{endpos}(p)$ 中 $\leq r$ 的最后出现位置 x ，然后判断是否有 $l \leq x - \text{minlen}(p) + 1$ 。

这样容易发现若转移结点都存在，则转移依然成立。复杂度多了一只 $\log$ 。

- 树结构上，子串 SAM 有什么性质？

祖先关系和原来一样，而且子 SAM 等价类的出现构成一个含根连通块。所以不需要额外调整树结构。

- 部分 SAM: 对于每个 r 都有 $f(r)$, 要求对所有 $S[f(r) \cdots r]$ 建 GSAM。

类似区间子 SAM 的构建, 我们找到每个点 endpos 集合中 length 最大的一个然后判断即可。

- 扫描线 + LCT 维护区间子串信息: 如**多次询问区间本质不同子串**问题。

大致思路是用 LCT 维护 link 树, 然后保证每条实链上 $rb = \max \text{endpos}$ 一样。

其实这个东西也可以使用树剖线段树代替, 不再展开。

- 广义 SAM 中每个串包含结点数量和 $\mathcal{O}(L\sqrt{L})$: 证明。

例题

- P3804 【模板】后缀自动机 (SAM)

就是让我们找出每个等价类的 `endpos` 集合大小。

- P6139 【模板】广义后缀自动机 (广义 SAM)

求 n 个字符串中本质不同的子串数量。

BFS 建出 GSAM，再跑本质不同子串数即可。

- P4770 [NOI2018] 你的名字

给定一个字符串 S ， Q 次询问，每次询问给出串 T 和 l, r ，求有多少 T 的非空子串不是 $S[l, r]$ 的非空子串。

$$1 \leq |S| \leq 5 \times 10^5, 1 \leq Q \leq 10^5, 1 \leq \sum |T| \leq 10^6.$$

先来看看前 68 分 $l = 1, r = n$ 怎么做。

不是子串 显然没有 是子串 更好做，考虑反着来。

这样问题转化为对 T 数其有多少子串在 S 中出现过，这不是子串匹配问题吗！

但是其实问题有点不太一样，我们要求的是 T 中有多少**本质不同**的子串在 S 中出现过。

实际上我们对 T 建出 SAM，对其每个等价类做上面的子串匹配问题。这可以使用 BFS 实现。

硬要加上 l, r 的限制就使用上面的区间子 SAM 的结构，每次看看能否转移即可。

注意自动机结构不是树，每个结点会被多次转移。所以 BFS 时要打个标记。

- P4022 [CTSC2012]熟悉的文章

给定 m 个模板串和 n 个匹配串，如果一个字符串是一个模板串的子串且长度不小于 L 则称其为“熟悉的”。

对于每个匹配串，求一个最大的 L ，满足将匹配串分割，熟悉的子串的总长度大于原串长度的 90%。

输入字符串总长不超过 10^6 。

显然考虑二分答案。

二分完了设一个 DP, $dp(i)$ 为以 i 结尾的前缀的最小不熟悉长度。

显然有 $dp(i) \leftarrow dp(i - 1) + 1$ 作为平凡的转移。

设当前匹配到 i 时最长匹配长度是 d , 若 $d \geq L$, 那么可以用 $\min_{i-d \leq x \leq i-L} dp(x)$ 去更新 $dp(i)$ 。

转移用单调队列维护, 匹配用 GSAM 实现。

- 一个 ix35 博客里看到的题：

给一个初始为空的字符串 S ，三种操作。

- 首部插入；
- 尾部插入；
- 给字符串 T ，询问 T 在 S 中出现了多少次。

做法是把它看成了两个栈，对中间一个长度 $2|T|$ 的字符串暴力匹配。

实际上这个做法配上重构，用后缀平衡树可以 $2 \log$ 解决 头/尾 插入/删除 四种操作。

（实际上无论怎么改，都可以用根号分治 + hash 解。）

- P5161 WD与数列

WD 很喜欢数列。他认为两个序列 A, B 是匹配的，当且仅当 $|A| = |B|$ 且对于 $1 \leq i, j \leq |A|, A_i - B_i = A_j - B_j$ 。

即长度相同且一个数列同时加上一个数可以和另一个数列完全一样。

现在 CX 给了他一个长度为 n 的大数列，WD 希望知道，数列中有多少对不相交的子串使得他们是匹配的。

$$1 \leq n \leq 3 \times 10^5。$$

作差分序列，不难发现这个条件与差分数组相同等价。

考虑建出 SAM，讨论 link 树中的一些性质，易得答案为：

$$\sum_{i,j} \min\{|i - j| - 1, \text{len}(\text{LCA}(i, j))\}$$

这个东西已经被转化为树上问题了，用 DSU on tree 配合线段树合并就可以做了。

这道题启发我们，字符串问题可以通过 SAM 转化为图论、树论问题。

- CF700E Cool Slogans

给定一个字符串 S ，要求构造字符串序列 $s_1, s_2, \dots, s_k$ ，满足任意 s_i 都是 S 的子串，且任意 $i \in [2, n]$ ，都有 s_{i-1} 在 s_i 中出现了至少 2 次（可以有重叠部分，只要起始、结尾位置不同即可）。

求可能的最大的 k 的值（即序列的最大可能长度）。

$n \leq 2 \times 10^5$ 。

显然我们要让字符串长度尽可能小，这样一定不劣。这样能够得到每个 s_{i-1} 都是 s_i 的 Border，且恰好出现 2 次。

贪心貌似做不太来，那就 DP。由于 Border 不太好处理，我们放松条件，只需要 s_{i-1} 是 s_i 的后缀即可。

这样对于同一个串，合法的转移就是一条祖先链。

由于状态是**不可区分**的，所以得到一个性质：

如果 p 在 link 树上是 q 的祖先，那么 p 状态包含的所有子串在 q 中的出现 endpos 集合相同。

这个性质没有特别显然，但还是挺好反证的。

利用这个性质，我们可以把 DP 状态只设在每个 SAM 的状态上。

现在我们已经可以二分一个祖先再判定做到 $\mathcal{O}(n \log^2 n)$ 了。

如果想要 $\mathcal{O}(n \log n)$ 需要一个性质：每条祖先链上 DP 值都是一段一段的，每段之间相差 1。

这样类似于双指针，每次只需要判断一个点的值。

- CF1098F Ж-function

给出一个字符串 s ，定义函数 $f(l, r)$ 为 $s[l, r]$ 的每个后缀与 $s[l, r]$ 的 lcp 之和。
现在询问 q 次，每次给出 l, r ，请输出 $f(l, r)$ 。

$$|s| \leq 2 \times 10^5.$$

求区间 Z 函数之和？

其实这个东西就是求 $\sum_{i=l}^r \text{LCP}(l, i)$ ，就是后缀树上的 $\sum_{i=l}^r \text{len}(\text{LCA}(l, i))$ 。

这个东西非常经典，设 $\Delta(i) = \text{len}(i) - \text{len}(\text{link}(i))$ ，离线拆询问做完了。

等等，真的是这样吗？我们发现还有一个限制是 r ，所以答案其实是

$\sum_{i=l}^r \min(\text{len}(\text{LCA}(l, i)), r - i + 1)$ 。

（下面用 $x \subseteq y$ 表示 x 在 y 的子树内）

我们考虑 u 的父亲边对询问 $[l, r]$ 的贡献。

由于 $\min(x, y) = \sum_{i=1}^x [i \leq y]$, 所以

$\text{len}(\text{LCA}(l, i)) \leq r - i + 1 \Leftrightarrow i \leq r - \text{len}(\text{LCA}(l, i)) + 1$, 可得贡献就是:

$$\sum_{v \subseteq u} [v \in [l, r - \text{len}(u) + 1]]$$

总答案是：

$$\begin{aligned} & \sum_{l \subseteq u} \sum_{v \subseteq u} [v \in [l, r - \text{len}(u) + 1]] \\ &= \sum_{l \subseteq u} \sum_{v \subseteq u} ([v \in [1, r - \text{len}(u) + 1]] - [v \in [1, l - 1]]) [l \leq r - \text{len}(u) + 1] \\ &= \sum_{l \subseteq u} [\text{len}(u) \leq r - l + 1] \sum_{v \subseteq u} [v \leq r - \text{len}(u) + 1] - \sum_{l \subseteq u} [\text{len}(u) \leq r - l + 1] \sum_{v \subseteq u} [v \leq l - 1] \end{aligned}$$

先看减号后面那个。第一个条件在一条祖先链上是单调的，可以倍增跳到一个 u_0 扔掉，然后做法就简单了。

关键是减号前面的式子。我们可以把它化成：

$$\sum_{l \subseteq u} [\text{len}(u) \leq r - l + 1] \sum_{v \subseteq u} [v + \text{len}(u) \leq r + 1]$$

这是一个二维偏序，可以直接完成。

E. 后缀平衡树

问题描述为每次在开头加入一个字符，并维护当前的后缀数组。

不难发现每次后缀数组最多加一个位置，所以可以用平衡树维护。

插入时有比较当前后缀和原来一个后缀的操作，可以直接计一个实数比较。

注意一下这个东西的实现，最好在重量平衡树上每个结点记一个值域，**左右旋时整棵子树暴力重构。**

和替罪羊树类似，复杂度是对的。~~不如直接写替罪羊~~

代码挺长的，懒得分页了。大家爱看不看。

```
#include<bits/stdc++.h>
using namespace std;
#define f(x,a,b) for(int x = a; x <= b; x++)
#define fr(v,s,b) for(int v = s; v >= b; v--)
#define cl(a,v) memset(a, v, sizeof(a))

typedef double db;

const int N = 1000010;

int Q, fir, lst; char inp[3 * N], s[N];
db val[N]; rnd[time(0)];

struct node {
    int lc, rc, id, siz, dat;
    db lb, rb;
} t[N]; int rt, tot;

inline bool cmp(int x, int y) { // check if x > y
    return (x[x] == s[y] ? val[x - 1] > val[y - 1] : s[x] > s[y]);
}

inline void update(int p) {
    if(p == 0) return;
    t[p].siz = t[t[p].lc].siz + t[t[p].rc].siz + 1;
}

void zig(int &p) {
    int q = t[p].lc; t[p].lc = t[q].rc, t[q].rc = p;
    update(p), update(q), p = q;
}

void zag(int &p) {
    int q = t[p].rc; t[p].rc = t[q].lc, t[q].lc = p;
    update(p), update(q), p = q;
}

void rebuild(int p, db l, db R) {
    if(p == 0) return;
    t[p].lb = l, t[p].rb = R, val[t[p].id] = (l + R) / 2.0;
    rebuild(t[p].lc, l, (l + R) / 2.0);
    rebuild(t[p].rc, (l + R) / 2.0, R);
}

void insert(int &p, int v, db l, db R) {
    if(p == 0) {
        p = ++tot, t[p].lb = l, t[p].rb = R, val[v] = (l + R) / 2.0;
        return t[p].id = v, t[p].dat = rnd(), t[p].siz = 1, void();
    }
    if(cmp(v, t[p].id)) {
        insert(t[p].rc, v, (l + R) / 2.0, R), update(p);
        if(t[t[p].rc].dat < t[p].dat) zag(p), rebuild(p, l, R);
    }
    else {
        insert(t[p].lc, v, l, (l + R) / 2.0), update(p);
        if(t[t[p].lc].dat < t[p].dat) zig(p), rebuild(p, l, R);
    }
}

void del(int &p, int v) {
    if(t[p].id == v) {
        double l = t[p].lb, R = t[p].rb;
        if(t[p].lc && t[p].rc) {
            if(t[t[p].lc].dat < t[t[p].rc].dat)
                zig(p), del(t[p].rc, v), rebuild(p, l, R);
            else zag(p), del(t[p].lc, v), rebuild(p, l, R);
        }
        else {
            if(t[p].lc) zig(p), del(t[p].rc, v), rebuild(p, l, R);
            else if(t[p].rc) zag(p), del(t[p].lc, v), rebuild(p, l, R);
            else p = 0;
        }
    }
    else val[t[p].id] < val[v] ? del(t[p].rc, v) : del(t[p].lc, v);
    update(p);
}

int len;
bool check(int id) { // id >= 0
    fr(1, len, 1) if(!inp[i] || s[id - (len - 1)])
        return t[id - (len - 1)] > inp[1];
    return true;
}

int query(int p) {
    if(p == 0) return 0;
    if(check(t[p].id)) return query(t[p].lc);
    return t[t[p].lc].siz + 1 + query(t[p].rc);
}

inline void pushback(char v) {
    lst++, s[lst] = v, insert(rt, lst, 0, len);
}

void decode(int mask) {
    len = strlen(inp + 1);
    fo(1, len, 1)
        mask = (mask * 131 + i) % len;
    swap(inp[1 + 1], inp[mask + 1]);
}

int main() {
    cin >> Q, scanf("%s", inp + 1), len = strlen(inp + 1);
    fr(1, len, 1) inp[i + 1] = inp[i];
    len += 2, inp[2] = 'A' - 1, inp[3] = 'Z' + 1;
    fir = 1; fo(1, len, len) pushback(inp[i]);

    int mask = 0;
    while(Q--) {
        char ch[10]; scanf("%s", ch + 1);
        if(ch[1] == 'Q') {
            scanf("%s", inp + 1), decode(mask);
            int ans = query(rt); inp[1]++;
            ans = query(rt) - ans, mask ^= ans;
            printf("%d\n", ans);
        }
        else if(ch[1] == 'A') {
            scanf("%s", inp + 1), decode(mask);
            fo(1, len, len) pushback(inp[i]);
        }
        else {
            int x; scanf("%d", &x);
            while(x) del(rt, lst), val[lst] = 0, lst--, x--;
        }
    }
    return 0;
}
```